2. Testbench for I/O file transfer

Aim

To design and simulate a Verilog testbench that reads input data from one text file and corresponding valid bits from another text file, and writes the data into an output text file only when the valid signal is high.

Software Required

- Xilinx Vivado Design Suite
- Vivado Simulator (XSIM)
- Any text editor (Notepad++, VS Code, etc.) for creating input files

Procedure

1. Create Input Text Files

- Create data_in.txt containing input data values (one per line).
- Create valid.txt containing valid bits (0 or 1 per line).

2. Create a New Project

- Open Xilinx Vivado and create a new RTL project.
- Select the target FPGA device/board.
- Add a new Verilog source file and write the file-I/O testbench code.

3. Write the Testbench

- Use $fopen to open the two input files and one output file.
- Use $fscanf to read data and valid bits line-by-line.
- Generate a clock inside the testbench.
- On each clock cycle, check the valid signal.
- Use $fwrite to write data into the output file only when valid = 1.

4. Run Simulation

- Set the testbench as the top module.
- Run Behavioral Simulation.
- Allow the simulation to complete file reading and writing.

5. Verify Output

- Open the generated data_out.txt file.
- Confirm that only the data corresponding to valid = 1 has been written.

Testbench program for file I/O transfer

```
`timescale 1ns/1ps
module tb_file_filter;
   reg clk;
   reg [7:0] data_in;
   reg       valid;
   integer fd_data, fd_valid, fd_out;
   integer r1, r2;
   // Clock generation
   always #5 clk = ~clk;
   initial begin
      clk = 0;
      data_in = 0;
      valid   = 0;
      // Open files
      fd_data  = $fopen("I:\\Saveetha\\SOC\\Lab experiments\\data_in.txt", "r");
      fd_valid = $fopen("I:\\Saveetha\\SOC\\Lab experiments\\valid.txt",   "r");
      fd_out   = $fopen("I:\\Saveetha\\SOC\\Lab experiments\\data_out.txt", "w");
      // Read files line by line
      while (!$feof(fd_data) && !$feof(fd_valid)) begin
         r1 = $fscanf(fd_data,  "%d\n", data_in);
         r2 = $fscanf(fd_valid, "%d\n", valid);
         @(posedge clk);
         // Write only when valid = 1
         if (valid == 1'b1) begin
            $fwrite(fd_out, "%d\n", data_in);
         end
      end
      // Close files
      $fclose(fd_data);
```

```
        $fclose(fd_valid);
        $fclose(fd_out);
        $display("File processing completed.");
        #10 $finish;
    end
endmodule
```

Input

Data_in	101	102	103	104	105	106	107	108	109	110	111	112
Valid	1	0	1	0	1	0	1	0	1	0	1	0

Output

Data_out	101	103	105	107	109	111

Result

The Verilog testbench was successfully simulated in Vivado. The program correctly read input data and valid bits from two separate text files. Data samples were written to the output file only when the valid signal was high. The output text file contained only the filtered valid data, confirming the correct operation of file reading, conditional processing, and file writing in Verilog.